ScamShield PK

Digital Fraud Triage Expert System

Student: Muhammad Sarim Umer Roll no: ROLL-NO
Course: Artificial Intelligence | Type: rule-based expert system
Knowledge base: 161 rules, 48 facts, 18 scam types.

PLACEHOLDER -- Streamlit Community Cloud URL

Live app URL: _____
GitHub repo (public): _____
Deploy file: app.py Branch: main
Check from a phone / incognito window that the link opens.

How to fill this in:
1. Push this folder to a public GitHub repository.
2. Open https://share.streamlit.io -> New app.
3. Pick the repo, branch main, main file app.py.
4. Copy the public https://....streamlit.app URL here and into README.md.

1. What this program is

This is a rule-based expert system, not a chatbot and not a machine-learning model. Expert knowledge about Pakistan-specific digital fraud (fake JazzCash agents, BISP SMS, OLX overpayment, courier links, SIM-swap, fake NADRA, and others) is stored as IF-THEN rules with certainty factors. The engine matches the user's ticks against those rules and explains every conclusion.

Five classic expert-system parts:

- Knowledge base -- rules.py (data, not if/else)
- Inference engine -- engine.py (forward chaining + CF)
- Working memory -- the known dictionary
- User interface -- app.py (Streamlit)
- Explanation facility -- fired-rule list + family-tree diagram

2. Why forward chaining and certainty factors

Facts are known up front (what the user observed), so we chain forward from data to conclusions. Backward chaining would be for 'prove this goal' questions.

Certainty factor of one rule:

```
rule_cf = min(CF of each condition) * rule['cf']
```

Two rules supporting the same conclusion (MYCIN-style combine):

```
new = old + rule_cf * (1 - old)
```

Risk rules R51-R54 use first-match conflict resolution (critical, then high, medium, low). Flag, scam-type, and action rules fire all matches.

3. Five layers (the family tree)

- Layer 0 FACTS -- checkboxes the user ticks
- Layer 1 FLAGS -- credential_request, impersonation, ...
- Layer 2 SCAM TYPES -- wallet_fraud, sim_swap_scam, ...
- Layer 3 RISK -- critical / high / medium / low
- Layer 4 ACTIONS -- what to do right now

If no layer-2 rule matches, the engine does not invent a scam type. It records a knowledge gap (trace id GAP), sets risk Low, and tells the user to verify officially. Missing knowledge is not the same as 'safe'.

4. File-by-file code explanation

rules.py -- The knowledge base. FACTS, FLAGS, SCAM_TYPES, ACTIONS, HELPLINES, and RULES (a list of dictionaries). Each rule has id, layer, if, then, cf, why. Adding a new scam means adding a dict here; engine.py does not change. validate_knowledge_base() runs at import so a typo in a condition name crashes loudly instead of silently doing nothing in the viva.

engine.py -- infer(facts) is the whole reasoner. clean_facts() drops unknown ids and duplicates. Phase A forward-chains layers 1-2 until a full pass fires nothing (each rule at most once). scam_score is the max CF among scam types. Phase B-1 picks the first matching risk rule. Phase B-2 fires every matching action. Returns (known, fired) so the UI can show Why?

```
def infer(facts):
    known = {f: 1.0 for f in clean_facts(facts)}
    # Phase A: while changed, fire unfired layer 1-2 rules
    known['scam_score'] = max(scam-type CFs)
    # if scam_score == 0: mark no_matching_rule (GAP)
    # Phase B-1: first matching risk rule
    # Phase B-2: all matching action rules
```

tree.py -- Builds Graphviz DOT strings. case_tree_dot is the current case. full_tree_dot is the whole KB. scam_subtree_dot is one scam type for the PDF figures.

app.py -- Four tabs: Analyze, Knowledge Base, Full Tree, How it works. Load example writes session_state before widgets render (Streamlit rule). Clear resets ticks. Results stay on screen until you analyze again. Knowledge Base can filter by layer and search.

scenarios.py / tests/test_engine.py -- Demo cases double as pytest. The original 10 assignment scenarios must still pass. Extra cases cover utility bills, SIM-swap, crypto, charity, and NADRA.

student.py -- Name and roll number for the footer. Replace ROLL-NO before submission.

5. How to run locally

```
pip install -r requirements.txt
streamlit run app.py
```

Tests: pip install -r requirements-dev.txt then pytest

Rebuild this PDF: python scripts/build_pdf.py

Rebuild tree PNGs (needs Graphviz): python scripts/export_trees.py

6. Viva answers (short)

- Expert system = knowledge as rules + engine that explains itself.
- Forward chaining because facts are given first.
- Not plain if/else: rules are data; the Why? trace lists which rule fired.
- Limitation: hand-written rules, honest ticks, uncovered patterns look Low.
- Future: Roman-Urdu keywords, learned CFs, backward-chaining mode.

7. Knowledge sources (cite in the report)

- FIA NR3C public awareness (helpline 1991)
- PTA consumer awareness on SMS/call fraud
- SBP / bank pages: never share OTP/PIN
- JazzCash 4444, Easypaisa 3737, BISP 8171, NADRA 1515
- OLX Pakistan safety tips (overpayment)

Re-verify helpline numbers before you submit.